

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct heap {
    int c; // capacity
    int s; // size
    int *element;
};
typedef struct heap *pqueue;
pqueue initialize(int max) {
    pqueue heap1;
    if (max <= 3) {
        printf("\n Priority queue is too small\n");
        exit(EXIT_FAILURE);
    }
    heap1 = (pqueue)malloc(sizeof(struct heap));
    if (heap1 == NULL) {
        printf("\n Out ofspace\n");
        exit(EXIT_FAILURE);
    }
    heap1->element = (int *)malloc((max + 1) * sizeof(int));
    if (heap1->element == NULL) {
        printf("\n Out ofspace\n");
        exit(EXIT_FAILURE);
    }
    heap1->c = max;
    heap1->s = 0;
    heap1->element[0] = 0;
    return heap1;
}
int isempty(pqueue heap1) {
    return (heap1->s == 0);
}
int isfull(pqueue heap1) {
    return (heap1->s == heap1->c);
}
void insert(int x, pqueue heap1) {
    int i;
    if (isfull(heap1)) {
        printf("\n Heap is full\n");
        return;
    }
    for (i = ++heap1->s; heap1->element[i / 2] > x; i = i / 2)
```

```

heap1->element[i] = heap1->element[i / 2];
heap1->element[i] = x;
}
int deletemin(pqueue heap1) {
int i, child, min, last;
if (isempty(heap1)) {
printf("\n Heap is empty\n");
return heap1->element[0];
}
min = heap1->element[1];
last = heap1->element[heap1->s--];
for (i = 1; i * 2 <= heap1->s; i = child) {
child = i * 2;
if (child != heap1->s && heap1->element[child + 1] < heap1->element[child])
child++;
if (last > heap1->element[child])
heap1->element[i] = heap1->element[child];
else
break;
}
heap1->element[i] = last;
return min;
}
void display(pqueue heap1) {
int i;
printf("\nElements in the heap: ");
for (i = 1; i <= heap1->s; i++) {
printf("%d ", heap1->element[i]);
}
printf("\n");
}
int main() {
int s, ch, ele;
pqueue heap1;
printf("\nEnter the size of the queue: ");
scanf("%d", &s);
heap1 = initialize(s);
do {
printf("\nMenu\n1. Insert\n2. Delete\n3. Display\n4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &ch);
switch (ch) {
case 1:
printf("\nEnter the element to insert: ");

```

```
scanf("%d", &ele);
insert(ele, heap1);
break;
case 2:
ele = deletemin(heap1);
printf("\nThe deleted element is %d\n", ele);
break;
case 3:
display(heap1);
break;
case 4:
printf("\nExiting...\n");
break;
default:
printf("\nInvalid choice\n");
}
} while (ch != 4);
free(heap1->element);
free(heap1);
return 0;
}
```